

# DOSSIER S1 — CADRAGE & ARCHITECTURE

---

## FinData Solutions — Pipeline Big Data Hadoop

Module DEPE855 | EPSI Mastère Expert en Ingénierie des données

**Révision v2** — Remplacement de Sqoop par Apache NiFi (Flux 1), streaming Kafka + Spark Structured Streaming (Flux 3), couche Hive sur les données de production.

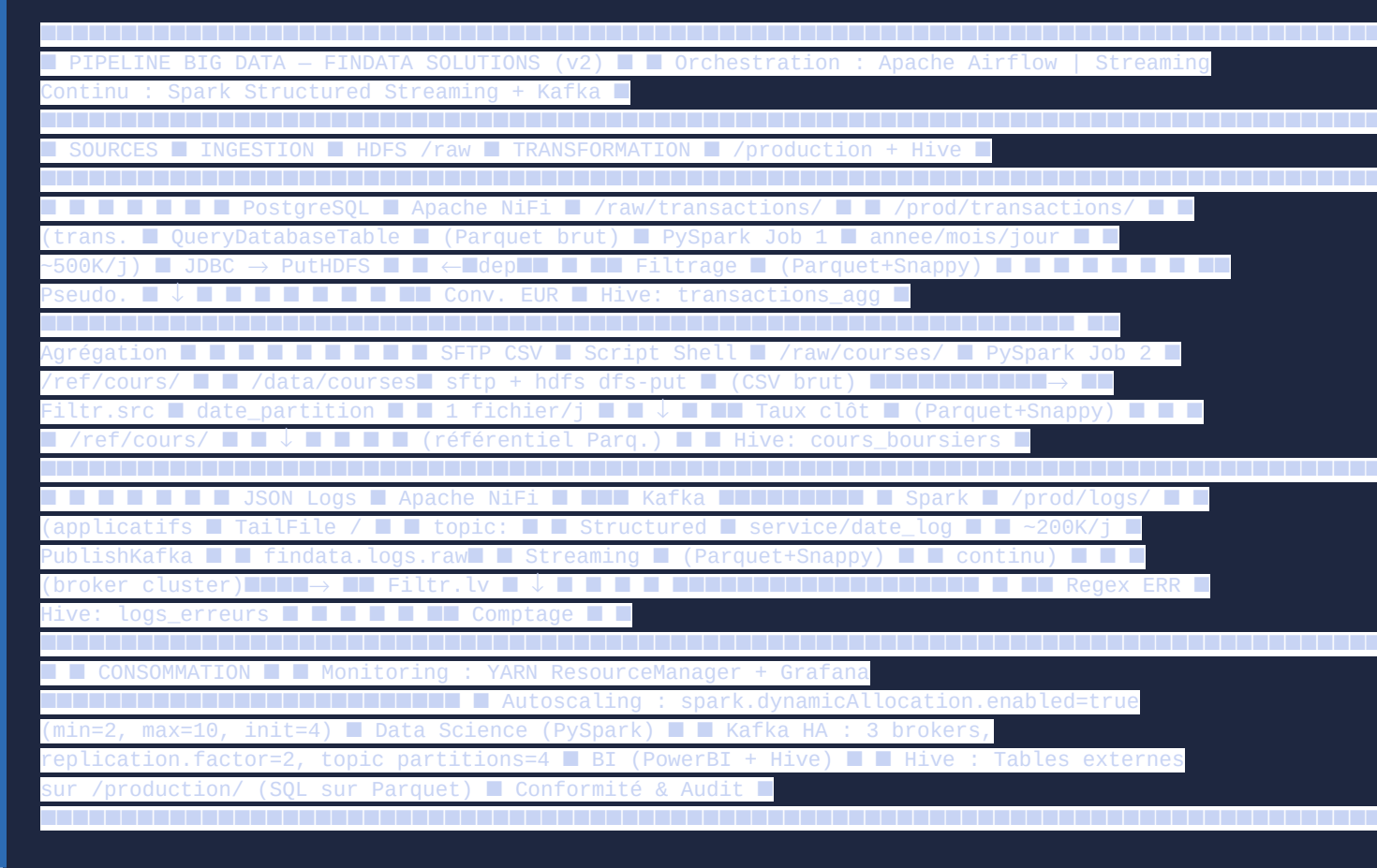
## 1. MATRICE DES BESOINS

| Flux                            | Source technique                                                           | Fréquence                                                          | Volume estimé                           | Équipe(s) destinataire(s)                                  | Format de sortie attendu                                 | Partitionnement HDFS | Contrainte de sécurité                                                                                  |
|---------------------------------|----------------------------------------------------------------------------|--------------------------------------------------------------------|-----------------------------------------|------------------------------------------------------------|----------------------------------------------------------|----------------------|---------------------------------------------------------------------------------------------------------|
| Flux 1 – Transactions bancaires | PostgreSQL, table transactions (mise à jour quotidienne)                   | Quotidienne (extraction J, lancée à 06h30 via NiFi)                | ~500 000 lignes/jour (~2 Go brut)       | Data Science, B.I., Conformité & Audit                     | Parquet (Snappy) — agrégé par code_banque et jour        | /production/         | Pseudonymisation : iban → SHA-256 + sel fixe ; client_id → identifiant séquentiel anonyme. RGPD + DSP2. |
| Flux 2 – Cours boursiers        | Fichiers CSV déposés via SFTP (/data/courses/avant-AAAA-MM-JJ.csv)         | Quotidienne (dépôt avant 06h00, avant Flux 1)                      | ~500 à 1 000 lignes/fichier (~50 Ko)    | Référentiel interne pour la conversion de devises (Flux 1) | Parquet (Snappy)                                         | /ref/cours/          | Filtrage source : ECB, Bloomberg, Reuters uniquement.                                                   |
| Flux 3 – Logs applicatifs       | Flux JSON semi-structuré des applications (NiFi → Kafka → Spark Streaming) | Continue / temps réel — traitement en micro-batches de 30 secondes | ~200 000 événements/jour (~500 Mo JSON) | Équipe technique, Data Science                             | Parquet (Snappy) — comptage erreurs par service et heure | /production/         | Aucune DCP ; filtrage ERROR/CRITICAL uniquement.                                                        |

**Dépendance critique** : Le Flux 2 doit être traité (zone **/ref/cours/**) **avant** le Flux 1 (conversion devises).

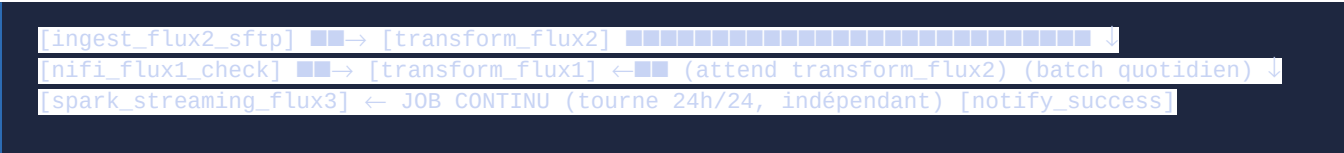
**Couche Hive** : des tables externes Hive sont créées sur chaque zone de production pour permettre des requêtes SQL aux équipes Data Science et B.I. sans connaissance de HDFS.

## 2. SCHÉMA GLOBAL DU PIPELINE (DIAGRAMME DE FLUX)



### Zones HDFS définies

| Zone        | Chemin                                                                           | Rôle                                                  | Rétention |
|-------------|----------------------------------------------------------------------------------|-------------------------------------------------------|-----------|
| Raw         | <a href="#">/raw/transactions/</a> ,<br><a href="#">/raw/courses/</a>            | Données brutes<br>post-ingestion, non<br>transformées | 7 jours   |
| Référentiel | <a href="#">/ref/cours/</a>                                                      | Taux de change de<br>clôture validés                  | 5 ans     |
| Production  | <a href="#">/production/transactions/</a> ,<br><a href="#">/production/logs/</a> | Données transformées,<br>pseudonymisées               | 5 ans     |
| Checkpoints | <a href="#">/checkpoints/logs/</a>                                               | Checkpoints Spark<br>Structured Streaming<br>(Flux 3) | Permanent |



### 3. JUSTIFICATION DES CHOIX TECHNIQUES

#### Ingestion — Flux 1 : Apache NiFi (remplace Sqoop)

**Choix retenu** : Apache NiFi avec processor `QueryDatabaseTableRecord` (connexion JDBC PostgreSQL).

**Pourquoi NiFi plutôt que Sqoop ?**

| Critère                 | Apache Sqoop                                       | Apache NiFi                                     |
|-------------------------|----------------------------------------------------|-------------------------------------------------|
| Maintenance             | Fin de vie (dernière release 2017, archivé Apache) | Activement maintenu                             |
| Interface               | Ligne de commande uniquement                       | Interface graphique + monitoring des flux       |
| Gestion des erreurs     | Redémarrage manuel                                 | Retry automatique, Dead Letter Queue            |
| Flexibilité du flux     | Un seul type de transformation                     | Routage conditionnel, enrichissement en vol     |
| Cohérence avec le reste | Outil différent de NiFi Flux 3                     | Un seul outil d'ingestion pour Flux 1 et Flux 3 |
| Format de sortie        | Parquet natif via <code>--as-parquetfile</code>    | Via <code>ParquetRecordSetWriter</code>         |

#### Processors NiFi pour le Flux 1 :

- `QueryDatabaseTableRecord` : requête JDBC sur PostgreSQL, filtrée par `DATE(date_heure) = '${date}'`
- `UpdateAttribute` : nommage du fichier de sortie `transactions_${date}.parquet`
- `PutHDFS` : écriture dans `/raw/transactions/date=${date}/`

**Justification du remplacement** : Sqoop est un outil legacy dont le développement est arrêté. NiFi offre une meilleure observabilité (chaque FlowFile est traçable), une gestion d'erreurs native, et permet une standardisation de l'outillage d'ingestion (un seul outil pour Flux 1 et Flux 3).

## Ingestion — Flux 3 : Apache NiFi + Apache Kafka (nouveau)

**Choix retenu** : NiFi collecte les logs JSON et les publie dans un topic Kafka. Spark Structured Streaming consomme le topic en temps réel.

### Pourquoi Kafka ?

- **Découplage** : NiFi (producteur) et Spark (consommateur) évoluent indépendamment. Si Spark s'arrête pour maintenance, Kafka conserve les messages (rétention configurable).
- **Scalabilité** : le topic `findata.logs.raw` est partitionné en 4 partitions → 4 consommateurs Spark parallèles.
- **Résilience** : replication factor = 2 → tolérance à la panne d'un broker.
- **Rejouer l'historique** : Kafka permet de repositionner l'offset pour rejouer les logs d'une fenêtre temporelle.

### Processors NiFi pour Kafka (Flux 3) :

1. `TailFile` ou `GetFile` : lecture des fichiers JSON depuis `/var/log/findata/apps/`
2. `SplitJson` : découpage en FlowFiles individuels (1 log = 1 message)
3. `PublishKafkaRecord_2_6` : publication dans le topic `findata.logs.raw`

---

## Transformation — Flux 3 : Spark Structured Streaming

**Choix retenu** : Spark Structured Streaming (API DataFrame) en lieu et place du job batch `flux3_logs.py`.

### Avantages vs batch :

- Latence de traitement : **30 secondes** (micro-batch) vs **24h** (batch quotidien)
- Détection quasi temps réel des cascades d'erreurs (ex: 50 ERR-3001 en 5 minutes)
- API identique à Spark SQL/DataFrame → réutilisation du code existant
- Écriture incrémentale en HDFS + mise à jour de la table Hive en continu

---

## Couche Hive (nouveau)

**Choix retenu** : Apache Hive avec tables **externes** sur les données Parquet de production.

### Pourquoi Hive ?

- **Accès SQL** pour les équipes B.I. et Data Science sans connaissance de HDFS
- **Tables externes** : Hive ne possède pas les données (la suppression de la table ne supprime pas les fichiers HDFS)
- **Partitionnement auto** : `MSCK REPAIR TABLE` ou `ALTER TABLE ADD PARTITION` découvrent automatiquement les nouvelles partitions HDFS
- **Connecteurs** : PowerBI se connecte à HiveServer2 via JDBC/ODBC sans accès direct à HDFS
- **Metastore** : catalogue centralisé des schémas, utilisable aussi par Spark (`spark.sql.catalogImplementation=hive`)

Orchestration : Apache Airflow (inchangé)

Le DAG est mis à jour pour remplacer la tâche Sqoop par un check NiFi (le job NiFi tourne en continu). Le job Spark Streaming Flux 3 est un service long-running démarré séparément.

Format de fichier : Parquet + Snappy (inchangé)

Choix confirmé. Compatible avec Hive, PowerBI et PySpark.

4. DATASETS SÉLECTIONNÉS

| Flux   | Dataset                             | Générateur           | Volume         | Fichier                    |
|--------|-------------------------------------|----------------------|----------------|----------------------------|
| Flux 1 | Transactions bancaires synthétiques | generate_datasets.py | 550 000 lignes | test_data/transactions_202 |
| Flux 2 | Taux de change EUR simulés          | generate_datasets.py | 800 lignes     | test_data/courses_2024-01- |
| Flux 3 | Logs JSON synthétiques              | generate_datasets.py | 220 000 lignes | test_data/logs_2024-01-15. |

Pour simuler le streaming Kafka en test : le script generate\_datasets.py produit les logs, puis un script les injecte dans Kafka via kafka-console-producer.

5. RÉPONSES AUX QUESTIONS DE RÉFLEXION (S1)

Question 1 — Pourquoi le Flux 2 doit-il être exécuté avant le Flux 1 ? Gestion de l'absence du fichier CSV

Réponse :

Le Flux 1 applique une règle de **conversion de tous les montants en EUR** en joinant avec le référentiel /ref/cours/. Si ce référentiel est absent ou vide pour la date du jour, la colonne montant\_eur sera NULL sur toutes les transactions en devises étrangères, rendant les agrégats incorrects.

Mécanisme de gestion dans Airflow :

```
# DAG : transform_flux1 attend explicitement transform_flux2 [transform_flux2, nifi_flux1_check] >> transform_flux1
```

En cas d'absence du fichier CSV SFTP :

- 1. Le script d'ingestion retourne exit code 1 → retry Airflow (3×, délai 5 min)

2. Si toujours absent après 3 tentatives : tâche `use_fallback_cours` copie le référentiel J-1
3. Le Flux 1 tourne avec les taux de la veille, données taguées `taux_source=Fallback`
4. Alerte envoyée à l'équipe et enregistrée dans les métadonnées Hive

Avec NiFi (vs Sqoop) : NiFi peut lui-même gérer ce wait via un processor `Wait` ou une condition `RouteOnAttribute(flowFile.exists == false)` qui déclenche une notification avant de router vers le fallback.

---

## Question 2 — La pseudonymisation SHA-256 avec sel fixe est-elle réversible ? Risques et alternatives

### Réponse :

SHA-256 est **non réversible mathématiquement**. Cependant, l'espace des IBANs français est fini et structuré (`FR76` + 23 chiffres). Avec un sel fixe unique pour tous les enregistrements, un attaquant qui connaît le sel peut construire une **rainbow table** et retrouver tous les IBANs.

### Risques si le sel est compromis :

- Réidentification complète de l'historique 5 ans (violation RGPD, amende jusqu'à 4% du CA)
- Non-conformité DSP2 sur la protection des données de paiement

### Alternative recommandée — Tokenisation via HashiCorp Vault (Transit Engine) :

- L'IBAN est envoyé au Vault qui retourne un token opaque aléatoire
- Seul le service Conformité & Audit peut demander la réversibilité au Vault
- La clé est stockée dans le Vault (jamais dans le code)
- Rotation de clé possible sans impacter les données pseudonymisées

---

## Question 3 — Avantage du format Parquet par rapport à CSV pour les requêtes ad hoc de la cellule B.I.

### Réponse :

Parquet est un format **columnar** : pour une requête `SELECT code_banque, SUM(montant_eur)`, seules ces 2 colonnes sont lues sur le disque (au lieu des 8 colonnes du CSV complet).

### Avantages clés pour la B.I. :

| Critère                                                    | CSV              | Parquet                                  |
|------------------------------------------------------------|------------------|------------------------------------------|
| Requête <code>SELECT 2 colonnes / 8</code> sur 500K lignes | Lit 100% (~2 Go) | Lit ~25% (~500 Mo)                       |
| Filtrage <code>WHERE annee=2024 AND mois=1</code>          | Scan complet     | Partition pruning → -97% de données lues |
| Compression                                                | Aucune           | Snappy → -80% de taille sur le disque    |

|                              |             |                                                |
|------------------------------|-------------|------------------------------------------------|
| Typage                       | Tout String | Types natifs (Double, Long, Timestamp)         |
| Compatibilité PowerBI + Hive | CSV lent    | Connecteur Parquet + HiveServer2<br>JDBC natif |

**Exemple chiffré** : requête B.I. taux de fraude sur janvier 2024 → CSV : ~8 min ; Parquet partitionné : ~25 secondes (gain ×19).

Avec Hive, la B.I. exécute directement `SELECT code_banque, nb_fraudes/nb_transactions FROM findata.transactions_agg WHERE annee=2024 AND mois=1` via JDBC sans connaissance de HDFS.